ProcRosetta: A Behavior-Supervised Multimodal Foundation Model for Process Mining

Alessandro Berti[1*] and Wil M. P. van der Aalst[1,2]

[1*]Process and Data Science Group, RWTH Aachen University, Ahornstrasse 55, Aachen, 52074, Germany.
[2]Celonis, Munich, Germany.

*Corresponding author(s). E-mail(s): a.berti@pads.rwth-aachen.de;
Contributing authors: wvdaalst@pads.rwth-aachen.de;

Abstract

Process-mining systems describe the same process through event logs, process trees, and Petri nets, yet learned representations are usually tied to one artifact type. We present ProcRosetta, a deterministic multimodal model that maps all three artifacts into a shared 96-dimensional semantic space and decodes any representation into a grammar-valid process tree. Its corpus comprises 15,360 synthetic behavior families (61,440 aligned rows) spanning three structural curricula, with 12,288 families used for fitting. Each family couples two Petri-net realizations with two clean log views and records exact-language or structural-isomorphism evidence. A 6.26-million-parameter architecture combines a prefix Transformer, a trace/set encoder, an edge-aware graph encoder, and a shared cross-attentive tree decoder. On fixed, family-complete test audits of 16 families per curriculum, fused-latent distance correlates increasingly with held-out behavioral distance from simple to complex processes (Spearman $\boldsymbol{\rho = 0.577}$, **0.743**, and **0.791**), and cross-modal family Recall@1 ranges from **0.656** to **0.875**. All 768 audited deployment decodes terminate as syntactically valid, Petri-convertible, duplicate-free trees. Exact reconstruction is nevertheless weak—zero for log inputs and at most **34.4%** for tree inputs—and classical Inductive Miner remains substantially better for discovery conformance. These results support a shared process representation and reliable constrained generation, but not replacement of established discovery algorithms. We release the generator, evaluation pipeline, exact provenance, and submission evidence bundle for reproducible scrutiny.

Keywords: process mining, foundation models, multimodal representation learning, process trees, Petri nets, event logs, grammar-constrained decoding

1 Introduction

Process mining connects observed event data to executable process models. Its core artifacts are heterogeneous: an event log is a finite multiset of sequences, a process tree is a hierarchical expression, and a Petri net is a typed graph with token-flow semantics (van der Aalst et al. 2004; van der Aalst 2016). Their syntax differs even when their visible behavior is intended to be the same. Consequently, learning systems are often built in modality-specific silos: one encoder for traces, one method for graph similarity, or one discovery network translating a log into a model. A representation learned in one silo cannot be used directly to retrieve, compare, fuse, or translate artifacts in another.

Foundation models seek reusable representations that support multiple downstream tasks and forms of input (Bommasani et al. 2021). In process mining, this idea is attractive but technically delicate. The training signal cannot be based on textual or graphical similarity alone. Two structurally different Petri nets may be behaviorally equivalent; finite logs provide incomplete observations; concurrency can share complete visible traces with explicit interleaving while differing in partial-order semantics; and loops make exact language enumeration impossible in general. A useful process foundation representation must therefore distinguish an artifact's presentation from its behavioral content, while retaining enough source detail for structured generation.

We address this problem with ProcRosetta, a multimodal representation and translation model for event logs, process trees, and Petri nets. Three specialized encoders produce six memory tokens and one normalized deterministic semantic vector per artifact. A single grammar-constrained Transformer decoder maps source memory back to a process tree. Training is organized around *behavior families*: groups that share a canonical behavior but vary in Petri realization and log observation. Exact certificates are used for strong positives; bounded or structurally motivated relations contribute only softer geometric supervision. This separation prevents an unverified equivalence from being treated as a hard identity.

The study asks:

RQ1: Does the learned space align modalities, remain stable across equivalent views, and order pairs by behavioral distance?
RQ2: Can each modality, and their fusion, be decoded into valid and behaviorally faithful process trees?
RQ3: How useful are the embeddings for cross-modal retrieval and how does log-to-tree decoding compare with classical process discovery?

The contributions are fourfold. First, we introduce a certificate-aware corpus generator with ordinary random trees and three controlled equivalence motifs, producing 15,360 family groups and 61,440 aligned rows across simple, medium, and complex curricula. Second, we develop a compact 6.26-million-parameter architecture in which modality-specific memories and a shared semantic vector serve both retrieval and constrained translation. Third, we train the representation with reconstruction, exact contrastive, hierarchical, observation-consistency, and behavioral-geometry objectives,

Table 1 Material changes from the July 2026 preprint to this journal submission.

Dimension	Preliminary preprint	This article
Training unit	Independently generated aligned examples	Grouped behavior families with four views and explicit certificates
Data coverage	One synthetic regime	Three curricula; 15,360 families; controlled motifs and ordinary cases
Latent model	Variational 64-dimensional representation	Deterministic, normalized 96-dimensional semantic representation
Encoders	Recurrent tree/log components and graph encoder	Prefix Transformer, biGRU plus set Transformer, and edge-aware residual GNN
Decoder	Recurrent generation	Three-layer cross-attentive Transformer with factorized grammar heads and activity copy
Supervision	Reconstruction and pair alignment	Multi-source reconstruction, exact and hierarchical metric learning, view consistency, behavior regression/ranking, and anti-collapse terms
Deployment	Basic grammar masking	Bounded completion, source-alphabet masking, duplicate-free policy, and Petri conversion checks
Evaluation	Aggregate prototype results	Fixed family-complete audits, baselines, controlled strata, bootstrap intervals, and disclosed failure modes

without a variational latent or Kullback–Leibler term. Fourth, we provide an evidence-oriented evaluation that reports failure cases and family-bootstrap intervals alongside point estimates.

This article supersedes an early preprint of the project (Berti and van der Aalst 2026b). Table 1 makes the distinction explicit. The previous prototype used a smaller corpus and recurrent/variational components; the present work replaces the data model, architecture, objective, deployment decoder, checkpoint-selection protocol, and evaluation. The preprint is cited to make the development history transparent rather than presented as independent prior evidence.

The remainder of the article reviews related work (Section 2), defines the formal setting (Section 3), explains corpus generation (Section 4), architecture and training (Sections 5–6), presents the experimental design and results (Sections 7–8), and closes with use, limitations, and conclusions.

2 Background and related work

2.1 Process representations and discovery

An event log records completed executions but only partially samples the behavior of an underlying process. A process tree composes activities and silent steps through sequence, exclusive choice, parallelism, and loop operators. Its block structure permits

conversion into sound workflow nets. A Petri net expresses places, transitions, arcs, and markings directly and can represent non-block structures not expressible by the chosen tree grammar (van der Aalst 2016; Kourani et al. 2026).

Classical process discovery constructs a model from a log. Inductive Miner recursively detects cuts and returns a sound block-structured result (Leemans et al. 2013). We use the PM4Py implementation for conversion, simulation, and evaluation (Berti et al. 2023). Learned discovery instead estimates a model from examples; graph-neural discovery is one relevant direction (Sommers et al. 2023). Our task is broader than discovery: the same parameters encode three artifact types, support retrieval and geometry, and decode all of them into one exchange language. Inductive Miner remains the appropriate discovery baseline rather than a straw-man competitor.

Conformance combines several perspectives. Fitness measures whether observed traces can be replayed, while precision penalizes model behavior unsupported by the log (Adriansyah et al. 2013). Directly-follows summaries are computationally convenient but lose long-range and concurrency information (van der Aalst 2019). We therefore report both distributional behavioral distances and process-model conformance, and avoid treating any single metric as semantics in full.

2.2 Representation learning for process mining

Trace-encoding methods range from handcrafted features to sequence models; their assumptions and performance vary considerably by task (Tavares et al. 2023). Earlier work learned representations at activity, trace, log, and model levels (Koninck et al. 2018). Petri-net embeddings have also been learned from graph structure (Colonna et al. 2024), and graph distances can be compared with model-similarity measures (Dijkman et al. 2011). These approaches establish the value of vector representations but generally do not align raw logs, hierarchical trees, and marked Petri graphs in one behavior-supervised space.

A concurrent process-foundation direction uses in-context learning on event logs for predictive monitoring (Berti and van der Aalst 2026a). That setting centers on one input modality and prediction over cases. ProcRosettainstead aligns multiple artifact types and produces complete structured models. The two directions are complementary: a future system could combine event-level monitoring capabilities with artifact-level semantic translation.

2.3 Multimodal alignment and constrained generation

Contrastive multimodal learning demonstrates that paired observations can define a shared space without requiring identical encoders (Radford et al. 2021). Supervised contrastive objectives allow all members of a semantic class to act as positives (Khosla et al. 2020). Our classes are behavior families rather than image–text pairs, and certificate strength controls which relations may be used as exact positives. Variance and covariance regularization reduce collapse without imposing a probabilistic prior (Bardes et al. 2022).

Transformers provide the attention mechanism used in the tree encoder, set encoder, and decoder (Vaswani et al. 2017). The log encoder must additionally be

invariant to trace order, motivating a set-attention layer (Lee et al. 2019). For output, grammar-constrained decoding can guarantee syntactic structure even when model probabilities are imperfect (Geng et al. 2023). Our decoder extends this principle with explicit operator arities, source-activity masking, contextual copy scores, and deterministic completion when the search budget is exhausted.

3 Formal setting

Let $\mathcal{A}$ be a finite activity alphabet and τ an invisible activity. A trace is a word $\sigma \in \mathcal{A}^*$ and an event log L is a finite multiset over traces. A process tree T is generated by

$$T ::= a \mid \tau \mid \mathsf{SEQ}_k(T_1, \ldots, T_k) \mid \mathsf{XOR}_k(T_1, \ldots, T_k) \mid \mathsf{AND}_k(T_1, \ldots, T_k) \mid \mathsf{LOOP}(T_1, T_2), \tag{1}$$

where $a \in \mathcal{A}$ and $k \geq 2$. The implementation stores prefix tokens with explicit arity, so a left-to-right parser knows the number of open child slots. A labeled Petri net is a tuple $N = (P, U, F, \ell, m_0, m_f)$ with places P, transitions U, flow relation F, labels $\ell : U \to \mathcal{A} \cup \{\tau\}$, and initial/final markings m_0, m_f.

For artifact modality $m \in \{T, L, N\}$, an encoder E_m returns source memory $H_m \in \mathbb{R}^{6\times 192}$ and a semantic vector

$$\boldsymbol{z}_m = \frac{g_m(H_m)}{\|g_m(H_m)\|_2} \in \mathbb{S}^{95}. \tag{2}$$

The decoder D estimates a tree-token sequence conditioned on either H_m or a memory projected from a fused semantic vector. Fusion is deliberately parameter-free:

$$\boldsymbol{z}_{\text{fuse}} = \frac{1}{|S|} \sum_{m \in S} \boldsymbol{z}_m, \qquad S \subseteq \{T, L, N\}, \quad |S| \geq 2. \tag{3}$$

The vector is not renormalized before projection in the implementation; its norm consequently carries modality-agreement information.

Behavior families.

A family f consists of a canonical tree T_f, two Petri realizations $N_f^{(1)}, N_f^{(2)}$, and two clean log views $L_f^{(u)}, L_f^{(r)}$. Flattening their Cartesian pairing yields four rows. All rows share a split-local family identifier; only verified exact relations share an exact-language identifier. The intended equivalence relation concerns normalized visible complete traces, supplemented by a partial-order identifier. It does not imply bisimulation, equal probabilities, or identical performance characteristics.

For held-out geometry we use a behavioral distance

$$d_B(f, g) = \frac{1}{3}\left(\|p^f_{\text{var}} - p^g_{\text{var}}\|_1 + \|p^f_{\text{df}} - p^g_{\text{df}}\|_1 + \|p^f_{\text{len}} - p^g_{\text{len}}\|_1\right), \tag{4}$$

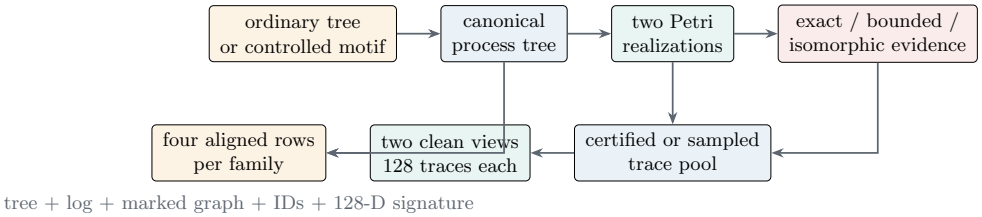

Fig. 1 Family-level generation. Family grouping is established before flattening, so all alternative realizations and observations remain in one data split.

where the terms are normalized trace-variant, boundary-aware directly-follows, and trace-length distributions. This is a finite-observation operationalization, not a complete semantic metric. Start/end, activity, eventually-follows, and graph-count features are retained as separate baselines.

4 Certificate-aware behavior-family corpus

4.1 Generation pipeline

The dataset is generated at the level of a latent *behavior family*, not one independently sampled row. Figure 1 gives the overview. All choices used for this study are specified below and in Algorithm 1; the version-7 manifest is the machine-readable record of the resulting splits, rather than a separate methodological specification.

One family is produced in six stages:

1. *Plan and random streams.* Motif and ordinary-root quotas are allocated before sampling. From global seed 13, a 64-bit BLAKE2b derivation of split name and family index creates a family seed; independent model, representation, playout, log-view, and noise streams are then derived from it. Within one candidate, consuming randomness for an observation therefore cannot advance the structural stream.
2. *Latent structure.* An ordinary family begins with a topology-first process-tree sample. Its root is an operator, arity is at most three, and the folded tree must meet the selected size, depth, and activity bounds. A controlled family instead begins with one of the three anchors in Section 4.2. The same randomized context is compiled around both anchor realizations.
3. *Paired models.* The folded tree is converted to a marked PM4Py workflow net. Ordinary families pair that net with an isomorphically renamed copy; controlled anchors provide two deliberately different Petri realizations. Both are serialized as typed graphs with place/transition type, visible label, directed arcs, and initial/final marking features.
4. *Acceptance and evidence.* Each variant must reach the final marking and have no dead transition in the bounded structural analysis. Non-loop families are compared by visible complete-trace enumeration; looping ordinary families use the stronger construction fact that the second net is an isomorphic clone of the first. The exact limits and meaning of each evidence tier are given below.

5. *Observed logs.* A shared clean trace pool is turned into two different 128-trace multisets: a deterministic coverage-oriented view and an independent resampled view. No trace corruption is applied in the persisted corpus used here.
6. *Canonicalize and flatten.* For each log view, activities are renamed $A0, A1, \ldots$ by first occurrence in that view; model activities absent from the log are appended in stable tree order. The same mapping is applied to the tree, traces, and Petri graph. Pairing two model variants with two log views yields four rows that remain in the same split.

This last convention matches external-log inference and prevents arbitrary source labels from becoming shortcuts. The vocabulary holds 30 visible activities. Every persisted `jsonl/process-sample.v5` row contains the three relabeled modalities, split-local family ID, exact/strong and partial-order IDs where justified, motif and complexity labels, variant/view IDs, source-specific decoder targets, a 128-dimensional behavior signature, certificate metadata, and all generation seeds.

4.2 Ordinary cases and controlled motifs

For an ordinary family, the generator repeatedly samples a topology (up to 4,000 attempts), assigns at least the profile's minimum number of distinct activities, reuses a prior activity with probability .15 thereafter, folds the result, and accepts it only if the folded size, depth, and alphabet lie inside the profile. The tree-derived workflow net is paired with a graph-isomorphic renamed copy. Non-looping cases are still enumerated rather than relying only on construction; looping cases are labeled `isomorphic` and use process-tree playout for their observations.

Three motifs introduce representation differences intentionally:

1. *Duplicate visible prefix versus silent routing*: the anchor tree is $\mathsf{XOR}(\mathsf{SEQ}(A0, A1), \mathsf{SEQ}(A0, A2))$. One net has two distinct transitions labeled $A0$; the other has one $A0$ followed by a silent left/right choice.
2. *True concurrency versus explicit interleaving*: the anchor $\mathsf{AND}(A0, A1)$ is represented once by true token concurrency and once by a progress-state net containing the explicit sequences $A0A1$ and $A1A0$. Their complete visible traces agree, but their partial-order IDs intentionally differ.
3. *Block versus non-free-choice M-pattern*: the anchor $\mathsf{XOR}(A0, \mathsf{AND}(A1, A2))$ is paired with the non-free-choice net in Figure 3. Both accept $\{A0, A1A2, A2A1\}$ before contextualization.

Each controlled anchor is substituted into one leaf of the same randomized binary SEQ/XOR/AND context for both realizations. The implementation draws two to four fresh context activities; including the motif placeholder, the context template therefore has 5, 7, or 9 nodes within the configured 4–12-node envelope. Capping the context at four added activities keeps an all-AND language enumerable under the default gates. This randomization prevents success based only on recognizing a canonical PM4Py conversion. Ordinary cases remain dominant: each training curriculum contains 3,056 ordinary, 347 duplicate-prefix, 347 concurrency, and 346 M-pattern families.

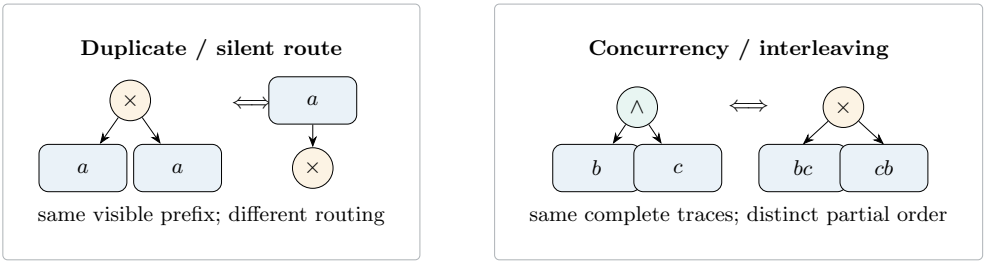

Fig. 2 Two controlled relations. The double arrow denotes equality of visible complete traces, not universal semantic equivalence.

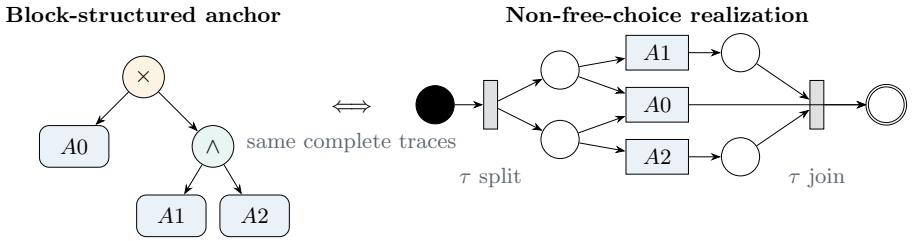

Fig. 3 The controlled M-pattern. Transition $A0$ competes for both post-split places, whereas $A1$ and $A2$ consume one each; this violates free choice while preserving the anchor's visible complete traces. Filled and double places denote the initial and final markings.

4.3 Certification, observation, and signatures

For every non-loop family, depth-first marking exploration enumerates the visible complete-trace language of each Petri variant. Search stops after 5,000 visited states, 10,000 completed traces, visible length 32, or a firing safeguard that also accounts for silent transitions. The variants are rejected if their collected languages differ or are empty. If the search exhausts both state spaces, the status is `exact`, and SHA-256 of the normalized language becomes the cross-family exact-behavior ID. If a cap is hit, equality applies only to the explored bound and the status is `bounded`; such a family cannot become an exact positive. Ordinary looping families skip this finite-language claim: their two nets differ only by an isomorphic renaming, receive status `isomorphic`, and derive their clean pool from 512 seeded process-tree playouts of maximum length 128. Training retries bounded candidates by configuration; validation and test would retain the weaker status if encountered. In the persisted corpus, all accepted families are either `exact` or `isomorphic`; no `bounded` family is used.

The trace pool for every other family is the sorted enumerated language, filtered to length at most 128. The `uniform_variants` view takes pool entries cyclically until it has 128 traces and then shuffles them; it balances all variants when the pool has at most 128 elements and takes the deterministic first 128 when the pool is larger. The `resampled` view makes 128 seeded draws with replacement. These are two observation multisets of one family, not two new process models. Although the generator schema

supports noisy, sparse, incomplete, and long-tail modes for separate studies, none is active in this corpus.

Finally, a family-level behavioral descriptor is computed from the normalized pool. Counts of trace length, start/end activities, activity occurrences, directly-follows and eventually-follows pairs, and complete variants are signed-hashed with BLAKE2b into 128 dimensions and ℓ_2-normalized. For the concurrency/interleaving analogue, a deterministic partial-order direction of strength .25 is added before re-normalization so equal complete languages are not mislabeled as equal causal structure. This signature provides soft geometry supervision; it is not an equivalence certificate.

4.4 Curricula, splits, and certificates

Table 2 consolidates the resolved generator settings and the three ordinary-tree profiles. The profile bounds apply to accepted folded ordinary trees; controlled anchors can be smaller than a curriculum's ordinary-tree target.

Table 2 Persisted generator configuration.

Component	Setting	Interpretation
Tree topology	arity ≤ 3; leaf probability .55; label-reuse probability .15	Labels are assigned after topology sampling so the activity floor is met without changing the root.
Operators	root: SEQ .70, XOR/AND/LOOP .10 each; non-root: .25 each	Root counts are quota-controlled for ordinary families; non-root operators are sampled.
Family mixture	ordinary .75; each controlled motif 1/12	Strict minima: 8 families/motif in training and 4 in validation/test.
Representations	2 Petri variants per family	Canonical conversion plus isomorphism, or the two motif-specific realizations.
Observations	2 clean views; 128 traces/view; trace length ≤ 128	Modes are `uniform_variants` and `resampled`; configured noise operators are inactive here.
Language search	5,000 states; 10,000 traces; visible length ≤ 32	Exhaustion yields exact evidence; hitting a limit yields only bounded evidence.
Format	schema 5; normalization `pm4py-fold-v1`	Generator/manifest version 7, activity vocabulary 30, global seed 13.

Profile	Recursion cap	Min. folded depth	Folded nodes	Visible labels
Simple	3	2	5–11	3–6
Medium	5	3	12–19	5–14
Complex	7	4	20–80	8–28

The simple, medium, and complex curricula are generated independently. Each has 4,096 training, 512 validation, and 512 test families; every family contributes four rows. Table 3 reports the actual held-out statistics recorded in the persisted manifest. The complete corpus therefore has 15,360 families and 61,440 rows, of which 12,288 families (49,152 rows) are used for fitting.

Table 3 Corpus design and actual test-family statistics. Size and depth refer to folded canonical trees; trace length is measured on stored observations.

Curriculum	Target size	Activity range	Train	Val.	Test	Mean size	Mean depth	Mean trace length
Simple	5–11	3–6	4,096	512	512	8.57	3.60	7.20
Medium	12–19	5–14	4,096	512	512	14.03	5.00	11.37
Complex	20–80	8–28	4,096	512	512	29.91	6.33	17.84
Total	–	–	12,288	1,536	1,536	–	–	–

Certificate metadata records the applied test, checked variants, reference-pool size, visible-length bound, normalization version, exact-language hash where available, and partial-order or structural-motif identifier. Strong positives require an exact-language identity plus compatible partial-order identity, or the within-family isomorphism construction used for looping ordinary cases. Controlled concurrency/interleaving pairs remain analogues rather than exact partial-order positives.

Family identifiers do not cross splits, so no realization or observation of one generated family leaks. However, independent generation does *not* guarantee that the same exact behavior or canonical tree cannot recur in another split. This matters: the evaluation is family-unseen, but not formally language-disjoint or tree-disjoint. We return to this limitation in Section 11.

5 Model architecture

5.1 Shared semantic interface

Figure 4 gives the end-to-end design. Each encoder returns a modality-specific memory used for reconstruction and a normalized semantic vector used for metric learning, retrieval, and fusion. A disposable two-layer projection head is used only by selected contrastive losses; reported embeddings and decoder conditioning always use the semantic vector itself.

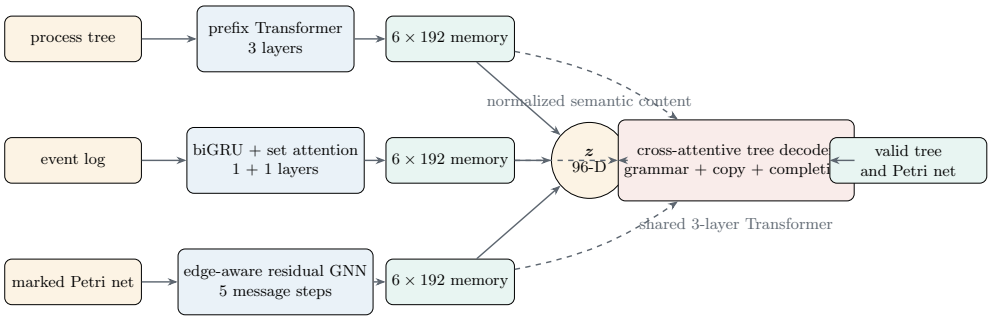

Fig. 4 ProcRosettaarchitecture. Solid paths identify the shared semantic interface and fused path; dashed paths show direct source-memory conditioning for modality reconstruction. Total trainable parameters: 6,263,248.

5.2 Process-tree encoder

The tree tokenizer linearizes Eq. (1) in prefix order and records token position, node depth, and parent position. The encoder sums learned embeddings for these four signals, applies three norm-first Transformer layers, and cross-attends six learned seed queries to the token sequence. This learned pooling preserves a small memory rather than collapsing immediately to one vector. A modality projection maps the pooled memory to Eq. (2).

5.3 Event-log encoder

For each trace, a bidirectional GRU encodes the activity sequence and event attention produces a trace representation. A one-layer Set Transformer then exchanges information among sampled traces without assigning semantic meaning to their presentation order. Six seed queries pool the set into memory. Contextual activity states are retained: all occurrences of an activity are aggregated and later scored by the decoder's copy component. During training, 32 of the 128 stored traces are resampled per presentation, with at most 64 events per trace.

5.4 Petri-net encoder

The Petri input is a typed directed graph. Initial node features combine place/transition type, visible transition label, and initial/final marking indicators. Five residual message-passing steps aggregate direction- and edge-type-aware messages over both arc directions. Each step uses normalization, and a jumping representation concatenates intermediate states before six-query pooling. Visible transition label states supply the decoder's Petri-side copy candidates.

5.5 Grammar-constrained decoder

The decoder has three Transformer layers with cross-attention at every layer. Its output distribution is factorized into structural token, operator arity, and visible-activity components. For activities, a learned gate combines a vocabulary distribution with a contextual copy distribution over source occurrences. This is important for external inputs: canonical labels can be generated because they occur in source memory even though their original names are restored only after inference.

At each search step, an automaton tracks open child slots. Illegal operators, arities, activities outside the source alphabet, and—under the deployment policy—already used visible activities receive zero probability. End-of-sequence is legal only when no slot remains. If beam search reaches its length budget with an open prefix, bounded completion repeatedly selects the shortest legal closure. The resulting token sequence is parsed, semantically folded, and converted to a Petri net. Syntactic validity, termination, duplicate-freedom, and Petri convertibility are recorded separately; none is used as a synonym for behavioral fidelity.

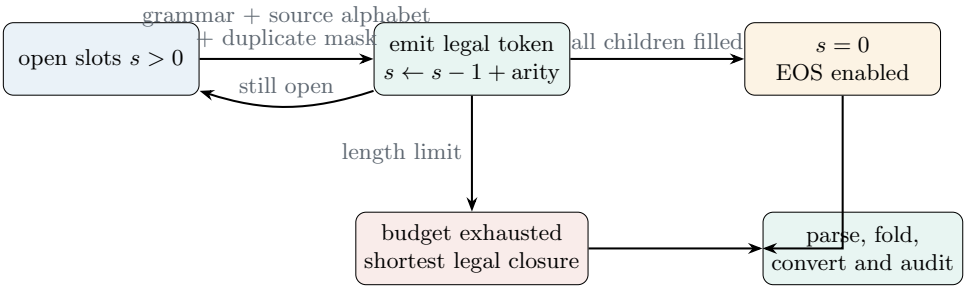

Fig. 5 Deployment decoding state. Bounded completion guarantees a syntactically closed prefix within the supported grammar; conversion and behavioral evaluation remain distinct postconditions.

Table 4 Architecture of checkpoint `proc_rosetta.best.pt`.

Component	Layers/steps	Parameters	Main dimensions
Tree encoder	3	1,953,534	token/position/depth/parent embeddings; 192 hidden
Trace encoder	1 GRU + 1 set layer	823,969	bidirectional trace state; six memory queries
Petri encoder	5	1,362,720	typed bidirectional messages; jumping states
Tree decoder	3	2,104,401	cross-attention; structural/arity/copy heads
Contrastive head	2-layer MLP	18,624	training-only projection
Total	–	6,263,248	six 192-D memory tokens; 96-D semantic vector

5.6 Capacity and implementation profile

Table 4 reports the persisted checkpoint architecture rather than nominal command-line defaults. The decoder and tree encoder contain most parameters; the contrastive head accounts for only 0.30% and can be discarded at deployment.

6 Training and checkpoint selection

6.1 Objective

Training minimizes a weighted sum grouped as

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{fuse}} + \mathcal{L}_{\text{exact}} + \mathcal{L}_{\text{hier}} + \mathcal{L}_{\text{obs}} + \mathcal{L}_{\text{geom}} + \mathcal{L}_{\text{reg}}. \tag{5}$$

Here $\mathcal{L}_{\text{recon}}$ is label-smoothed teacher-forced tree-token cross-entropy for all three sources; $\mathcal{L}_{\text{fuse}}$ reconstructs from the raw three-source mean and sampled two-source means. $\mathcal{L}_{\text{exact}}$ includes all-positive cross-modal and within-modal contrastive terms for certified identities, including a 4,096-entry cross-batch memory. $\mathcal{L}_{\text{hier}}$ orders exact,

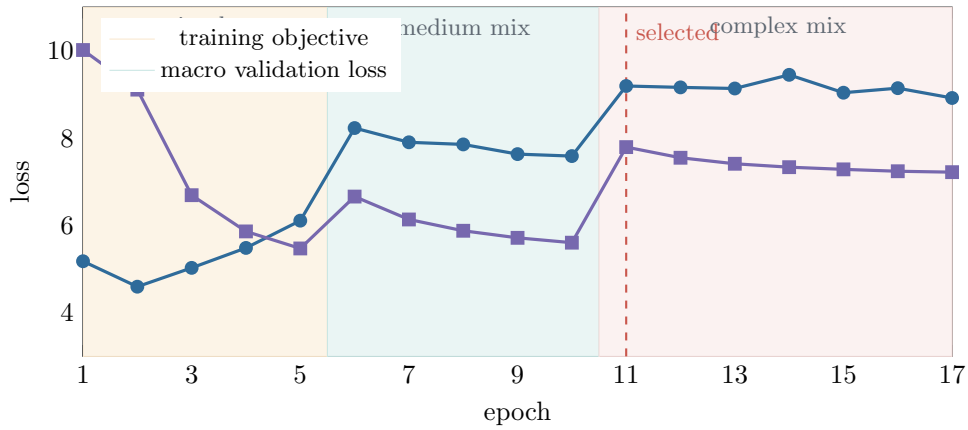

Fig. 6 Recorded training and macro validation objectives. Stage transitions change task difficulty, so loss is not expected to be monotone across epochs 5–6 and 10–11. The balanced checkpoint is epoch 11; training stops after epoch 17.

analogy, broader-family, and negative pairs. $\mathcal{L}_{\text{obs}}$ aligns alternate log views and simulated incomplete/noisy presentations. $\mathcal{L}_{\text{geom}}$ regresses and ranks observed behavioral distances and aligns cosine geometry to a soft 128-dimensional behavior signature. Finally, $\mathcal{L}_{\text{reg}}$ covers latent variance/covariance, positive alignment, termination, generation length, open slots, and completion.

This objective is deterministic. There is no sampled latent variable, no learned variance, and no Kullback–Leibler regularization. The full coefficient table is given in Appendix B; the largest individual reconstruction weights are 1.0, while the principal metric weights range from 0.10 to 0.30.

6.2 Optimization and curriculum

The model is optimized with AdamW at initial learning rate 3×10^{-4}, weight decay 5×10^{-4}, label smoothing 0.04, and gradient-norm clipping at 5. Dropout is 0.12 in the tree and Petri encoders and 0.20 in the trace encoder, decoder, and semantic projections. Activity identifiers are randomly permuted with probability 0.5, and two family views are selected in group-aware batches. Effective default batch sizes are 128, 96, and 64 for simple, medium, and complex curricula.

The realized training schedule differs from the nominal 100-epoch horizon because competence gates advanced early and early stopping fired. Epochs 1–5 use only simple families; epochs 6–10 sample approximately 25% simple and 75% medium; epochs 11–17 sample approximately 10% simple, 20% medium, and 70% complex. Reconstruction trains first, exact/hierarchical supervision ramps over epochs 3–6, and observed geometry ramps over epochs 5–10. An exponential moving average with decay 0.995 starts at epoch 3. Scheduled sampling begins at epoch 15 and had only begun its ramp when training stopped.

6.3 Validation and model selection

Validation loss is computed separately on all three curricula. Periodic fixed-subset audits additionally measure decoding, retrieval, family invariance, behavior geometry, and discovery. The robust selection score weights decoding 40%, retrieval 20%, equivalence 15%, geometry 15%, and discovery 10%, subject to decoding gates. Epoch 11 is the first complex-stage full audit and is retained as the balanced checkpoint. Specialized checkpoints for decoding, retrieval, geometry, discovery, and each curriculum are persisted, but all results below use the one predeclared balanced checkpoint. No test result selects a checkpoint.

The run records 17 completed epochs and 19,441.6 seconds (5.40 hours) of aggregate wall time. The checkpoint records CUDA execution but not the accelerator model; we therefore do not present hardware-normalized throughput claims.

7 Experimental design

7.1 Evaluation sample and reproducibility unit

The complete test split contains 512 families (2,048 rows) per curriculum. Routine decoding and discovery are expensive because every generated tree is converted and compared, so the persisted evaluation protocol selects a deterministic, stratum-interleaved set of 16 complete families (64 rows) in each curriculum. Family identifiers are ordered by a stable hash within complexity, motif, and observation strata; all four rows of every selected family are retained. The headline audit consequently covers 48 families, 192 rows, and four source conditions, for 768 deployment decodes. Claims in this article refer to this fixed audit unless explicitly labeled as corpus statistics. They are not extrapolated as full-test estimates.

Beam width is 2, maximum generation length is 512 tokens, and 16 traces are simulated for decoded-behavior comparisons. Deployment decoding constrains visible labels to the source alphabet and forbids duplicate visible labels. Tree-edit distance is normalized to $[0, 1]$; the behavioral distance in Eq. (4) can exceed 1 and has maximum 2. Exact reconstruction compares folded canonical token sequences.

Uncertainty is estimated by resampling complete families with replacement and recomputing the statistic; rows of a family are never split across replicates (Efron and Tibshirani 1994). The implementation uses 500 deterministic replicates for row-aggregated measures and 200 for the quadratic pairwise Spearman calculation. We report percentile 95% intervals for the principal geometry, retrieval, and discovery measures. With only 16 families per curriculum, the intervals are intentionally visible and no null-hypothesis significance claims are made.

7.2 Metrics for the research questions

For **RQ1**, within-family cosine similarity measures invariance across the two Petri representations and two log views. Mean cross-family similarity provides a global contrast; a nearest-positive minus nearest-negative margin provides a local contrast; and family Recall@1 tests whether the closest candidate shares the family. Geometry is evaluated by Spearman correlation between cosine distance and d_B. Baselines include

trace-variant, directly-follows, eventually-follows, activity, and trace-length/log features; Petri node/edge counts; and PM4Py's Node2Vec-style Petri embedding based on Grover and Leskovec (2016).

For **RQ2**, we report termination, grammar validity, Petri convertibility, duplicate freedom, exact tree match, normalized tree-edit distance, and d_B. These are non-interchangeable: constrained decoding can guarantee the first four but not semantic accuracy. We separately stratify the log decoder by motif and loops.

For **RQ3**, cross-modal retrieval uses each artifact as a query and the strongest available exact-behavior identifier as relevance; in every fixed audit this identifier is present and corresponds one-to-one with the 16 selected family identifiers (four rows each). Recall@1 and mean reciprocal rank (MRR) are reported. For discovery, the log-decoded tree and PM4Py Inductive Miner are compared with footprint fitness and precision, summarized by harmonic mean F1. Some grammar-valid generated trees are too large for PM4Py footprint extraction; evaluability is therefore reported rather than silently dropping failures.

7.3 Qualitative external case

The repository's running example is held outside the synthetic curriculum command used for the quantitative test. It has six traces and eight visible labels. We encode its PTML, XES, and PNML artifacts independently, decode each with a 128-token budget, restore original labels, and compare the resulting tree to the source. This is an illustrative case, not a statistical real-world benchmark.

8 Results

8.1 RQ1: the shared space captures family identity and behavioral order

Table 5 compares semantic embeddings with deterministic baselines. Fused geometry improves with structural complexity, from $\rho = 0.577$ on simple families to 0.791 on complex families. Its 95% family-bootstrap intervals are $[0.405, 0.833]$, $[0.593, 0.868]$, and $[0.675, 0.891]$. The simple trace-variant baseline is higher (0.713), showing that learning is not uniformly superior. Fused embeddings lead the listed baselines on medium and complex audits; on complex families, a rich deterministic PM4Py log vector is close at 0.757.

Figure 7 visualizes the principal comparison. It also cautions against a single-baseline narrative: direct behavior features are strong when the evaluation distance contains related statistics. The learned advantage is instead that one fixed-dimensional interface is available from all three modalities and supports translation and retrieval as well as distance.

Equivalent-view invariance is consistently high (Table 6). Family Recall@1 is 1.0 for every modality and curriculum in this audit. This task is intentionally within-modality: it asks whether alternate views/realizations are neighbors. Cross-modal retrieval is evaluated separately below. The local closest-positive minus closest-negative margin

Table 5 Spearman correlation between representation distance and held-out behavioral distance (ρ, higher is better). Bold indicates the highest value in a curriculum.

Representation	Simple	Medium	Complex
ProcRosetta fused semantic	0.577	**0.743**	**0.791**
ProcRosetta trace semantic	0.576	0.714	0.723
ProcRosetta Petri semantic	0.569	0.714	0.755
ProcRosetta tree semantic	0.433	0.576	0.680
Trace-variant distribution	**0.713**	0.628	0.453
Directly-follows distribution	0.550	0.693	0.750
PM4Py log feature vector	0.406	0.633	0.757
Eventually-follows features	0.289	0.503	0.599
Activity features	0.229	0.490	0.624
Petri count features	0.206	0.562	0.581
PM4Py Petri Node2Vec	0.189	0.201	0.535

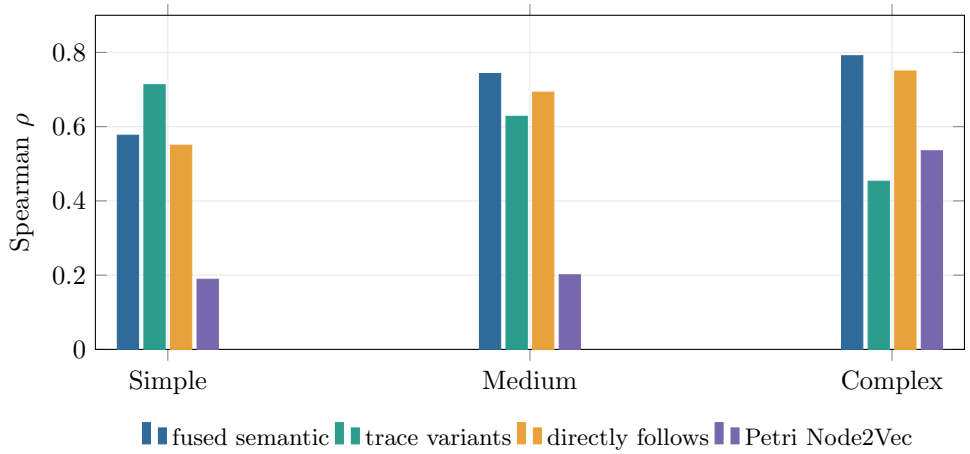

Fig. 7 Behavior-distance correlation. The fused learned representation strengthens across curricula; the trace-variant baseline is strongest on simple families, while directly-follows features remain competitive on complex families.

is smallest for the trace encoder because finite observations vary even within one behavior family.

The joint PCA in Fig. 8 is a qualitative projection of one row from 12 complex families. It explains 40.4% of full-space variance. Most same-family modality triples occupy a local neighborhood, but they are not collapsed to one point; the full-space mean same-row cross-modal cosine distance is 0.476 versus 0.962 for distinct-family pairs. This distinction explains why within-modality invariance can be near 1.0 while cross-modal retrieval is imperfect.

Table 6 Within-modality equivalence audit. "Within" and "between" are mean cosine similarities over same-family and cross-family pairs. Margin is the per-query closest-positive similarity minus closest-negative similarity.

Curriculum	Encoder	Within ↑	Between ↓	Margin ↑	Family R@1 ↑
Simple	Fused	.995	.227	.223	1.000
	Petri	.985	.220	.230	1.000
	Trace	.985	.316	.137	1.000
	Tree	.999	.163	.323	1.000
Medium	Fused	.993	.155	.287	1.000
	Petri	.985	.151	.298	1.000
	Trace	.976	.260	.156	1.000
	Tree	.999	.118	.370	1.000
Complex	Fused	.995	.101	.291	1.000
	Petri	.985	.103	.326	1.000
	Trace	.982	.221	.182	1.000
	Tree	.999	.065	.372	1.000

Answer to RQ1.

Within the fixed audit, ProcRosettaAlearns a stable family representation and a meaningful behavioral ordering. The evidence is strongest for within-modality view invariance and medium/complex fused geometry, but the wide simple-curriculum interval and strong direct-feature baselines preclude a claim of universal metric superiority.

8.2 RQ2: constrained validity is perfect, reconstruction is not

Every audited deployment output terminates, satisfies the prefix grammar, avoids duplicate visible labels, and converts to a Petri net: 768 of 768 decodes. Table 7 shows why validity must be reported separately from accuracy. Tree-input exact match declines from 34.4% to 15.6%; Petri input reaches at most 12.5%; log input never exactly reconstructs the canonical tree. The log encoder nevertheless maintains a normalized edit distance near 0.52–0.56 across curricula, suggesting structurally related but nonidentical outputs. Fused decoding does not outperform the best single modality.

Behavior simulation or footprint extraction can fail even after Petri conversion. The log-source behavior score is available for all simple and complex rows and 96.9% of medium rows; the fused complex score is available for 93.8%. Table values average evaluable outputs. These failures arise from very large, deeply nested but grammar-valid generated trees, sometimes approaching the 512-token budget. They are a model-quality and evaluator-tractability failure, not a syntax failure.

Motif stratification (Table 8) localizes the main degradation. Controlled fragments retain almost constant edit distance because their anchored structures are small. Ordinary processes become much harder with complexity; their mean log-decoder edit

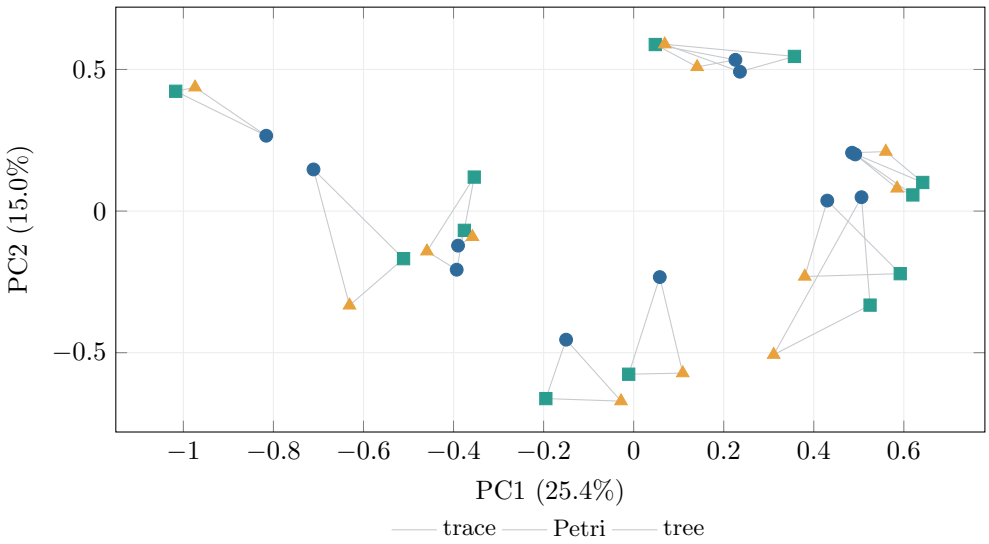

Fig. 8 Joint PCA of trace, Petri, and tree semantics for 12 complex families. Gray triangles connect modalities of one family. PCA is illustrative; all reported retrieval and distance results use the original 96-dimensional vectors.

distance grows from 0.662 to 0.832, and behavior distance from 1.224 to 1.715. Looping rows follow the ordinary stratum in this subset and are the primary source of long outputs.

Answer to RQ2.

The deployment wrapper reliably returns a valid executable representation, but the learned decoder is not a reliable inverse of the encoders. Grammar constraints solve well-formedness; they do not solve semantic reconstruction. The resulting model is suitable for candidate generation and inspection, not unattended model replacement.

8.3 RQ3: cross-modal retrieval is useful; discovery remains behind

Cross-modal family retrieval is substantially above the $4/64 = 1/16 = 0.0625$ random-row rate. Table 9 reports all six directions. Recall@1 ranges from 0.656 to 0.875 and mean MRR from 0.705 to 0.908. The hardest direction is log-to-tree; its Recall@1 is 0.656, 0.656, and 0.688, with bootstrap intervals $[0.438, 0.844]$, $[0.421, 0.844]$, and $[0.469, 0.844]$. Tree queries retrieve logs especially well (0.875 in every curriculum).

The discovery comparison is more sobering (Table 10). ProcRosetta's decoded models achieve mean footprint F1 around 0.65–0.67, and only 71.9–78.1% are evaluable by the footprint routine. Inductive Miner is evaluable for every family and achieves 0.938–0.996. The non-overlap of most bootstrap intervals reinforces a large practical gap, although paired significance is not claimed.

Table 7 Bounded duplicate-free deployment decoding. Valid includes termination, grammar validity, and Petri convertibility. Behavior and edit are lower better; exact is folded canonical equality.

Curriculum	Source	Valid (%)	Exact (%)	Behavior $d_B \downarrow$	Edit $\downarrow$
Simple	Tree	100	34.4	0.861	0.509
	Log	100	0.0	1.169	0.519
	Petri	100	12.5	1.094	0.661
	Fused	100	3.1	1.148	0.589
Medium	Tree	100	28.1	0.950	0.549
	Log	100	0.0	1.246	0.553
	Petri	100	6.2	1.130	0.645
	Fused	100	0.0	1.222	0.668
Complex	Tree	100	15.6	1.105	0.645
	Log	100	0.0	1.291	0.562
	Petri	100	6.2	1.245	0.673
	Fused	100	0.0	1.273	0.676

Table 8 Log-to-tree decoding by generator motif (behavior distance / normalized edit distance; lower is better).

Family type	Simple	Medium	Complex
Duplicate prefix	1.136 / .546	1.187 / .546	1.165 / .546
Concurrency/interleaving	1.087 / .491	1.081 / .491	1.063 / .491
M-pattern	1.175 / .378	1.170 / .378	1.130 / .378
Ordinary	1.224 / .662	1.580 / .798	1.715 / .832

This comparison clarifies what the shared model currently contributes. It provides one representation for heterogeneous inputs, supports bidirectional retrieval, and always returns a bounded syntactically valid candidate. It does not beat a specialized algorithm that directly exploits process-tree cut structure. A useful deployment would use ProcRosettato retrieve related artifacts, propose candidates, or initialize a downstream procedure, while retaining conformance validation and classical discovery.

Answer to RQ3.

The learned interface is useful for cross-modal retrieval on the fixed gallery, with consistent results in all six directions. As an end-to-end discovery system it is decisively inferior to Inductive Miner and suffers downstream evaluability failures on overlong candidates.

8.4 Running-example translation

Table 11 presents the external running example. All outputs use exactly the eight source labels, contain no extra or repeated visible labels, satisfy the grammar, and

Table 9 Cross-modal family retrieval as Recall@1 / MRR. Each candidate gallery contains 64 rows from 16 held-out families; the four same-family rows are relevant.

Direction	Simple	Medium	Complex
Petri → log	.703 / .755	.766 / .799	.828 / .855
Petri → tree	.828 / .862	.828 / .862	.828 / .862
Log → Petri	.750 / .805	.719 / .785	.750 / .801
Log → tree	.656 / .705	.656 / .705	.688 / .730
Tree → Petri	.812 / .867	.875 / .908	.875 / .908
Tree → log	.875 / .905	.875 / .905	.875 / .905

Table 10 Log-to-model discovery using footprint conformance. Intervals are 95% family-bootstrap intervals for F1.

Curriculum	Method	Evaluable (%)	Fitness	Precision	F1	95% interval
Simple	ProcRosetta	78.1	.702	.654	.666	[.598, .746]
	Inductive Miner	100	1.000	.992	.996	[.987, 1.000]
Medium	ProcRosetta	75.0	.709	.663	.669	[.611, .733]
	Inductive Miner	100	1.000	.986	.992	[.975, 1.000]
Complex	ProcRosetta	71.9	.691	.639	.649	[.570, .729]
	Inductive Miner	100	1.000	.907	.938	[.868, .991]

Table 11 External running-example translation with a 128-token budget. Conformance compares each decoded model with the six-trace source log.

Input	Output size	Depth	Edit ↓	Fitness	Precision	F1
Source reference	14	6	0.000	1.000	1.000	1.000
Tree	16	6	0.360	.474	.360	.409
Log	14	5	0.545	.737	.400	.519
Petri	14	5	**0.182**	**.860**	**.700**	**.772**

convert to Petri nets. Petri input gives the closest result (edit 0.182; footprint F1 0.772). The log output has lower structural fidelity but moderate footprint fitness. The tree autoencoding output is valid yet alters routing substantially, illustrating that even direct reconstruction should be reviewed.

The source tree is

```
SEQ(register request, LOOP(SEQ(AND(check ticket, XOR(examine thoroughly,
examine casually)), decide), reinitiate request), XOR(reject request, pay
compensation)).
```

The Petri-derived output preserves the central parallel and choice fragment but moves `reinitiate request` into the loop body:

```
SEQ(register request, LOOP(AND(check ticket, XOR(examine thoroughly, examine
casually)), SEQ(decide, reinitiate request)), XOR(reject request, pay
compensation)).
```

The example demonstrates readable cross-artifact translation, not identity. Complete source and output strings are included in Appendix D.

9 Using the representation

The released implementation exposes three levels of use. First, command-line utilities encode XES, PNML, and PTML files and print the semantic vector. Second, conversion utilities decode any supported artifact into PTML and a derived PNML. Third, the browser-based Multimodal Process Studio groups imported artifacts, compares cosine similarities, plots PCA neighborhoods, performs weighted exploratory fusion, and exports validation reports.

External labels are canonicalized internally and restored after decoding. The deployment policy constrains output to the complete source activity inventory and avoids duplicate visible labels. These safeguards prevent hallucinated activity names but also exclude legitimate repeated-label models. Users can inspect the raw semantic path separately, but quantitative headline results use the bounded duplicate-free path throughout.

A minimal reproducible workflow is:

```
./sample.py --data-dir data --train-families 4096 \
  --validation-families 512 --test-families 512 \
  --activity-vocab-size 30 --traces-per-sample 128
./train.py --data-dir data \
  --checkpoint checkpoints/proc_rosetta.pt \
  --epochs 100 --batch-size 128
./test.py --data-dir data --curriculum complex \
  --checkpoint checkpoints/proc_rosetta.best.pt --json
scripts/xes_to_ptml.py input.xes decoded.ptml \
  --checkpoint checkpoints/proc_rosetta.best.pt
```

The paper bundle adds the exact corpus manifest, checkpoint provenance, extracted result tables, and build instructions. Checkpoints are treated as trusted executable content and are not accepted from browser uploads.

10 Discussion

10.1 What has been demonstrated

The clearest result is that behavior-family supervision can coordinate three structurally different encoders. Equivalent views are stable within each modality, cross-modal retrieval is far above chance, and fused cosine distance orders complex held-out family pairs in agreement with an operational behavioral metric. This supports the core idea of a shared process representation.

The second result is architectural rather than predictive: the decoder and deployment wrapper separate *well-formedness* from *quality*. Grammar masks and completion

make every audited output executable in the narrow syntactic sense, but reconstruction and conformance expose large semantic errors. Reporting only valid-output rate would therefore be misleading. The distinction is relevant to any neural process-model generator.

Third, the arithmetic fused representation is a useful common interface but is not uniformly the best decoder condition. It improves geometry on harder curricula yet does not improve reconstruction. A learned reliability-aware fusion module may help when modalities differ in observation quality, but it would require ablation against the transparent raw mean.

10.2 Why representation quality exceeds generation quality

Metric learning only requires semantically related samples to occupy suitable neighborhoods; decoding requires recovery of a long discrete structure under teacher-forcing mismatch. Many trees can explain a finite log, while the training target chooses one canonical realization. Exact tree match is therefore a strict target, but the discovery results show that non-uniqueness is not the entire explanation: generated candidates also lose conformance and sometimes grow pathologically long.

The decoder's copy mechanism succeeds at vocabulary control, not necessarily at control-flow placement. The high penalty for an early structural error propagates through prefix generation because it changes all subsequent open slots. Scheduled sampling began only at epoch 15 and training stopped after epoch 17, leaving little exposure to self-generated prefixes. Future work should investigate stronger sequence-level risk objectives, explicit size priors, behavior-guided reranking, and staged decoder-only continuation while freezing a validated semantic space.

10.3 The meaning of "foundation model"

ProcRosettais a domain foundation *prototype*: one pretrained representation supports several modalities and tasks without task-specific encoders. It is not a general-purpose or large-scale foundation model in the contemporary language-model sense. It has 6.26 million parameters, a synthetic alphabet of 30 labels, and evidence on retrieval, geometry, and structured decoding rather than broad downstream adaptation. We retain the term to describe the shared reusable substrate, while the limitations below bound the claim.

11 Threats to validity

Synthetic-to-real generalization.

Training and quantitative testing use the same generator family. Controlled motifs increase structural diversity, but no industrial multi-artifact benchmark is evaluated. The one running example establishes file-level operation only. Noise, lifecycle attributes, resources, timestamps, concept drift, and real label semantics are absent.

Family-unseen is not behavior-unseen.

All representations of a generated family remain in one split, preventing direct row leakage. Independently generated families can nevertheless share an exact visible language or canonical tree across splits. The present results are therefore not a strict compositional or language-level generalization test. A future release should deduplicate exact and normalized structural signatures globally before splitting.

Finite behavioral semantics.

Equation (4) depends on sampled traces and contains trace-variant and directly-follows components also represented among training signatures and baselines. This makes it relevant to the optimized notion of behavior but not fully independent. Concurrency versus explicit interleaving shares complete traces while differing in partial orders; looping-family observations are finite playout samples rather than enumerated languages. No claim of bisimulation, stochastic-language equivalence, or general equivalence beyond the recorded evidence follows.

Evaluation scope and uncertainty.

The routine audit has 16 families per curriculum, selected deterministically rather than sampled anew. Bootstrap intervals are wide and quantify only resampling of those families, not generator, training-seed, or checkpoint uncertainty. A single training seed (13) is evaluated. Full 512-family tests and multiple retrainings are needed for precise performance estimates.

Decoder policy and metric availability.

Duplicate-free source-alphabet decoding narrows the legal hypothesis class and can inflate validity relative to unconstrained generation. Conversely, it excludes legitimate repeated-activity models. Footprint evaluation fails on some large valid outputs; conformance means are conditional on evaluability and are accompanied by the evaluated fraction. Behavior simulation uses only 16 traces per decode.

Baselines and ablations.

The embedding table includes strong deterministic features but not other learned multimodal process models because a directly comparable system was not identified. The discovery comparison uses Inductive Miner, not all discovery algorithms. No retrained loss/encoder/fusion ablation is available, so the contribution of individual objective terms is not causally identified. Architecture descriptions establish what was trained, not that every component is necessary.

Reproducibility environment.

The checkpoint stores software schema, seed, optimizer state, history, and CUDA use, but not the GPU model. Wall time is thus descriptive only. PM4Py behavior and conformance can change across versions; the environment lock and schema checks should be used when regenerating results.

Table A1 Controlled motif intent and evidence boundary in the persisted corpus.

Motif	Representation contrast	Relation retained	Explicit non-claim
Duplicate prefix	duplicate visible transitions vs shared visible transition plus silent routing	exact normalized visible language and shared partial-order ID	equal internal routing structure
Concurrency	true parallel fragment vs explicit sequence interleavings	exact visible-language ID but distinct partial-order IDs	equal concurrency or causal semantics
M-pattern	block-structured fragment vs non-free-choice realization	exact normalized visible language in the accepted corpus	universal equivalence under other contexts or bounds
Ordinary	canonical tree conversion vs isomorphic renaming	exact language for non-loops; within-family isomorphism for loops	an enumerated or cross-family exact-language ID for loops

12 Conclusion

We introduced ProcRosetta, a certificate-aware multimodal model that aligns event logs, process trees, and Petri nets in one deterministic semantic space and decodes each modality through a shared process-tree grammar. The new behavior-family corpus, architecture, training objective, and evidence protocol replace the project's preliminary preprint design.

The results establish a promising representation: family views are invariant, cross-modal Recall@1 is 0.656–0.875, and fused behavioral correlation reaches 0.791 on the complex audit. Constrained decoding is operationally reliable in that every audited output is closed, valid, duplicate-free, and Petri-convertible. Semantic generation is the limiting factor: log exact reconstruction is zero, fused decoding offers no consistent gain, overlong trees reduce evaluator coverage, and Inductive Miner remains far superior for discovery. ProcRosettashould therefore be treated as a retrieval and candidate-generation substrate with mandatory validation.

The next research step is not merely to scale parameters. It is to construct globally behavior-disjoint real and synthetic benchmarks, learn observation-aware fusion, preserve the validated semantic geometry during decoder improvement, and evaluate whether the representation transfers to conformance, discovery, retrieval, and monitoring tasks under limited supervision.

Appendix A Generation and certification details

Algorithm 1 summarizes one split. Sampling retries until all target family slots are filled and minimum motif coverage is met. A strict infeasible request fails before replacing an existing dataset.

The train motif quotas per curriculum are 3,056 ordinary, 347 duplicate-prefix, 347 concurrency, and 346 M-pattern families. Validation/test quotas are 376, 45, 46, and

Algorithm 1 Certificate-aware family generation

Require: curriculum profile c, family target n, deterministic seed s

initialize split-local family list $\mathcal{F} \leftarrow \varnothing$
allocate deterministic motif quotas and ordinary-root quotas
while $|\mathcal{F}| < n$ **do**
    select the next quota slot and derive family-specific random streams
    sample an accepted ordinary tree, or anchor a controlled motif in one shared context
    construct paired Petri realizations $N^{(1)}, N^{(2)}$
    reject failed conversion, final-reachability, or dead-transition checks
    if T is an ordinary looping tree **then**
        certify the paired construction as isomorphic
        simulate 512 seeded tree playouts and retain the unique trace pool
    else
        enumerate each visible language under the state/trace/length gates
        reject empty or unequal collected languages
        label the evidence exact if exploration exhausts; bounded otherwise
        use the sorted collected language as the clean trace pool
    end if
    derive 128-trace uniform-variant and resampled clean views
    first-seen relabel each view and flatten 2 variants × 2 views
    store family IDs, evidence, signature, seeds, targets, and four rows
end while
verify motif × representation coverage and atomically commit split

45. Each family yields two Petri variants by two log views, so flattened row counts preserve these proportions exactly.

Exact generation invocation.

The command below creates the three curricula and their family-level splits. Values not repeated on the command line are the versioned defaults in Table 2; the persisted manifest records the resolved configuration and post-generation audits.

```
./sample.py --data-dir data --train-families 4096 \
  --validation-families 512 --test-families 512 --seed 13 \
  --generator behavior_families --activity-vocab-size 30 \
  --traces-per-sample 128 --max-trace-length 128 \
  --variants-per-behavior 2 --log-views-per-behavior 2 \
  --log-view-modes uniform_variants,resampled
```

Appendix B Objective coefficients and hyperparameters

Table B2 reports the checkpoint configuration. Adaptive validation balancing can modestly change source reconstruction multipliers; at the selected epoch they are 1.000 (tree), 1.099 (trace), 1.064 (Petri), and 1.122 (fused).

Table B2 Principal objective coefficients in the selected checkpoint. Zero entries are shown to prevent obsolete variational claims.

Term	Weight	Term	Weight
Tree reconstruction	1.00	Trace reconstruction	1.00
Petri reconstruction	1.00	Fused reconstruction	1.00
Fused subset reconstruction	.25	Deployment reconstruction	.25
Exact cross-modal contrast	.30	Within-modality contrast	.25
Semantic exact contrast	.15	Cross-batch memory contrast	.10
Hierarchical metric ordering	.15	Observation consistency	.15
Soft behavior geometry	.25	Observed-distance regression	.20
Observed-distance ranking	.20	Latent alignment	.075
Variance regularization	.10	Covariance regularization	.01
EOS / length / slots / completion	.05 each	Duplicate-activity penalty	0
Kullback–Leibler	0	Tree-complexity penalty	0

Table B3 Training and decoding hyperparameters persisted with the balanced checkpoint.

Hyperparameter	Value	Hyperparameter	Value
Seed	13	Activity vocabulary	30
Hidden / semantic dimension	192 / 96	Memory tokens	6
Base batch size	128	Effective S/M/C batches	128 / 96 / 64
Stored / presented traces	128 / 32	Presented trace cap	64 events
AdamW learning rate	3×10^{-4}	Weight decay	5×10^{-4}
Gradient clip	5.0	Label smoothing	.04
Activity remapping	.50	Views per family	2
EMA decay / start	.995 / epoch 3	Memory bank	4,096
Scheduled sampling	max .20	Start / ramp	epoch 15 / 20 epochs
Test beam / budget	2 / 512	Behavior simulations	16

Appendix C Metric definitions and complete training history

Normalized tree-edit distance is the ordered tree-edit cost divided by the larger folded-tree size. Exact match is equality after parsing and semantic folding. The within-modality equivalence audit uses split-local family identifiers. Cross-modal retrieval uses the strongest exact-behavior identifier; for the fixed audits it partitions the 64-row gallery into the same 16 four-row groups. Cosine distance is $1 - \boldsymbol{z}_i^\top \boldsymbol{z}_j$ because individual modality semantics are unit normalized.

Footprint fitness and precision compare directly-follows, parallel, start, and end relations derived from a log with those of a process tree. Their harmonic mean is reported only if both are available. This differs from alignment-based conformance and should not be compared numerically across conformance definitions.

Table C4 Complete persisted epoch history. Selection scores are not directly comparable when the periodic full discovery audit is absent; the balanced checkpoint is chosen by the configured robust audit at epoch 11.

Epoch	Training loss	Validation loss	Selection score	Learning rate
1	5.179	10.018	.595	3.00×10^{-4}
2	4.596	9.111	.610	3.00×10^{-4}
3	5.028	6.693	.643	3.00×10^{-4}
4	5.484	5.863	.644	3.00×10^{-4}
5	6.109	5.472	.679	3.00×10^{-4}
6	8.231	6.661	.763	3.00×10^{-4}
7	7.904	6.136	.707	3.00×10^{-4}
8	7.855	5.878	.705	1.50×10^{-4}
9	7.633	5.716	.715	1.50×10^{-4}
10	7.588	5.605	.796	1.50×10^{-4}
11	9.194	7.794	.787	1.50×10^{-4}
12	9.162	7.550	.703	1.50×10^{-4}
13	9.137	7.412	.713	7.50×10^{-5}
14	9.449	7.334	.707	7.50×10^{-5}
15	9.040	7.284	.721	3.75×10^{-5}
16	9.144	7.241	.716	3.75×10^{-5}
17	8.919	7.220	.719	1.875×10^{-5}

Appendix D Complete qualitative outputs

The running-example source and decoded folded trees are listed below. Line breaks are typographic only.

Source.

```
SEQ(register request, LOOP(SEQ(AND(check ticket, XOR(examine thoroughly,
examine casually)), decide), reinitiate request), XOR(reject request, pay
compensation)).
```

Tree input.

```
SEQ(register request, LOOP(AND(examine casually, XOR(decide, AND(examine
thoroughly, reinitiate request))), SEQ(pay compensation, reject request)),
XOR(tau, check ticket)).
```

Event-log input.

```
SEQ(register request, XOR(examine casually, AND(check ticket, decide,
reinitiate request, LOOP(examine thoroughly, pay compensation))), XOR(tau,
reject request)).
```

Petri-net input.

```
SEQ(register request, LOOP(AND(check ticket, XOR(examine thoroughly, examine
casually)), SEQ(decide, reinitiate request)), XOR(reject request, pay
compensation)).
```

Acknowledgements. The authors thank the Process and Data Science group at RWTH Aachen University for the research environment in which this work was developed.

Declarations

Funding. Funded by the European Union. This work has received funding from the EuroHPC Joint Undertaking and BMFTR, MKW NRW, and HMWK under Grant Agreement No. 101250682. Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union or the granting authorities.

Competing interests. Wil M. P. van der Aalst is affiliated with Celonis. The authors declare no other financial or non-financial competing interests relevant to this work.

Ethics approval and consent to participate. Not applicable. The study uses synthetic process data and a public software running example and involves no human participants, personal data, or animals.

Consent for publication. Not applicable.

Data availability. The deterministic corpus generator, exact generation command, schema validators, and evaluation code are available at https://github.com/fit-alessandro-berti/proc-rosetta. The submission source bundle contains the exact 112-KB curriculum manifest and machine-readable evidence tables used in this article. The generated corpus is approximately 1.7 GB and can be reproduced from seed 13 with the command and resolved settings reported in this article.

Materials availability. Not applicable beyond the software, reproducible synthetic data, and manuscript evidence bundle described above.

Code availability. Source code, command-line tools, the Streamlit inspection application, Docker configuration, tests, and reproduction instructions are available in the repository above. The evaluated code revision is `52a92f09f07ddc6b9b30463c5216973db7c76f83`. The model checkpoint uses format version 7, architecture identifier `proc-rosetta-latent-transformer-v6`, data schema 5, and normalization `pm4py-fold-v1`.

Author contributions. Alessandro Berti: conceptualization, methodology, software, data curation, investigation, formal analysis, visualization, and writing—original draft. Wil M. P. van der Aalst: conceptualization, methodology, supervision, validation, and writing—review and editing. Both authors approved the submitted version and accept responsibility for the work.

Preprint disclosure. An earlier prototype was posted as a preprint (Berti and van der Aalst 2026b). This article cites that version and explicitly documents the material changes in Table 1; it does not reuse the obsolete preprint's architecture or numerical claims as evidence for the present model.

Generative AI statement. During manuscript preparation, the authors used OpenAI Codex to assist with repository auditing, LaTeX drafting, generation of TikZ figures and tables, consistency checks, and language editing. The authors inspected the underlying source code and artifacts, verified the reported calculations, claims, and references, revised the generated text, and take full responsibility for the content.

References

van der Aalst WMP (2016) Process Mining: Data Science in Action, 2nd edn. Springer, Berlin, Heidelberg, https://doi.org/10.1007/978-3-662-49851-4

van der Aalst WMP (2019) A practitioner's guide to process mining: Limitations of the directly-follows graph. In: Proceedings of CENTERIS/ProjMAN/HCist, Procedia Computer Science, vol 164. Elsevier, pp 321–328, https://doi.org/10.1016/j.procs.2019.12.182

van der Aalst WMP, Weijters T, Maruster L (2004) Workflow mining: Discovering process models from event logs. IEEE Transactions on Knowledge and Data Engineering 16(9):1128–1142. https://doi.org/10.1109/TKDE.2004.47

Adriansyah A, Munoz-Gama J, Carmona J, et al (2013) Alignment based precision checking. In: Business Process Management Workshops, Lecture Notes in Business Information Processing, vol 132. Springer, pp 137–149, https://doi.org/10.1007/978-3-642-36285-9_15

Bardes A, Ponce J, LeCun Y (2022) VICReg: Variance-invariance-covariance regularization for self-supervised learning. In: International Conference on Learning Representations

Berti A, van der Aalst WMP (2026a) An in-context foundation model for predictive process monitoring on event logs. IEEE Access https://doi.org/10.1109/ACCESS.2026.3658877

Berti A, van der Aalst WMP (2026b) ProcRosetta: A foundation model for multimodal process mining. Preprints.org, https://doi.org/10.20944/preprints202607.0463.v1, version 1, posted 7 July 2026

Berti A, van Zelst SJ, Schuster D (2023) PM4Py: A process mining library for python. Software Impacts 17:100556. https://doi.org/10.1016/j.simpa.2023.100556

Bommasani R, Hudson DA, Adeli E, et al (2021) On the opportunities and risks of foundation models. arXiv preprint arXiv:210807258 https://doi.org/10.48550/arXiv.2108.07258

Colonna JG, Fares AA, Duarte M, et al (2024) Process mining embeddings: Learning vector representations for petri nets. Intelligent Systems with Applications 23:200423. https://doi.org/10.1016/j.iswa.2024.200423

Dijkman RM, Dumas M, van Dongen BF, et al (2011) Similarity of business process models: Metrics and evaluation. Information Systems 36(2):498–516. https://doi.org/10.1016/j.is.2010.09.006

Efron B, Tibshirani RJ (1994) An Introduction to the Bootstrap. Chapman and Hall/CRC, https://doi.org/10.1201/9780429246593

Geng S, Josifoski M, Peyrard M, et al (2023) Grammar-constrained decoding for structured NLP tasks without finetuning. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, pp 10932–10952, https://doi.org/10.18653/v1/2023.emnlp-main.674

Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, pp 855–864, https://doi.org/10.1145/2939672.2939754

Khosla P, Teterwak P, Wang C, et al (2020) Supervised contrastive learning. In: Advances in Neural Information Processing Systems, pp 18661–18673

Koninck PD, vanden Broucke S, Weerdt JD (2018) act2vec, trace2vec, log2vec, and model2vec: Representation learning for business processes. In: Business Process Management, Lecture Notes in Computer Science, vol 11080. Springer, pp 305–321, https://doi.org/10.1007/978-3-319-98648-7_18

Kourani H, Park G, van der Aalst WMP (2026) A discovery technique for expressive yet sound process models. Process Science 3(1)

Lee J, Lee Y, Kim J, et al (2019) Set transformer: A framework for attention-based permutation-invariant neural networks. In: Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 97. PMLR, pp 3744–3753

Leemans SJJ, Fahland D, van der Aalst WMP (2013) Discovering block-structured process models from event logs: A constructive approach. In: Application and Theory of Petri Nets and Concurrency, Lecture Notes in Computer Science, vol 7927. Springer, pp 311–329, https://doi.org/10.1007/978-3-642-38697-8_17

Radford A, Kim JW, Hallacy C, et al (2021) Learning transferable visual models from natural language supervision. In: Proceedings of the 38th International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 139. PMLR, pp 8748–8763

Sommers D, Menkovski V, Fahland D (2023) Supervised learning of process discovery techniques using graph neural networks. Information Systems 115:102209. https://doi.org/10.1016/j.is.2023.102209

Tavares GM, Oyamada RS, Barbon S, et al (2023) Trace encoding in process mining: A survey and benchmarking. Engineering Applications of Artificial Intelligence 126:107028. https://doi.org/10.1016/j.engappai.2023.107028

Vaswani A, Shazeer N, Parmar N, et al (2017) Attention is all you need. In: Advances in Neural Information Processing Systems